

Topo-DiT: Balanced Routing Bottlenecks for Latent Video Flow Matching

Research proposal and executable prototype - revised 25 September 2026

Abstract

Long video sequences make dense transformer computation expensive. We investigate a latent-grid flow model that aggregates spatiotemporal features into a smaller set of feature slots using balanced entropic transport, processes these slots with a time-conditioned transformer, and broadcasts the resulting features back to the latent grid. The proposed contribution is a testable routing design, not a claim that compact tokens inherently represent physical objects. We specify consistent transport marginals, latent-space training and sampling, an optional masked reconstruction warmup, and matched routing ablations. The implementation passes 26 CPU-only correctness tests without video-model training. No video-generation quality, physical reasoning, memory advantage, or distributed-scaling result is claimed. The current artifact is intended to support subsequent controlled experiments under limited local compute.

1. Motivation and scope

A small set of intermediate tokens can concentrate expensive computation. However, sequence compression alone neither establishes object discovery nor proves an end-to-end efficiency gain. This work asks a narrower question: **does a balanced routing bottleneck improve the quality-compute tradeoff relative to otherwise matched unbalanced routing?**

We retain the project name Topo-DiT for continuity. “Topo” currently denotes a learned assignment structure, not a proven topological invariant or a physically grounded scene graph. Cognitive-workspace analogies are motivation only. Camera conditioning, causal streaming, physical attention masks, symbolic reasoning, and distributed execution are not implemented claims.

The scope of the executable method is a flow field on continuous video latents. The RGB tokenizer is a separate frozen component. Video frame index and flow time are different variables: frame index locates a position within a clip; flow time parameterizes transport from noise to a data sample.

2. Related work and differentiation

Object-centric representations. Slot Attention [1] extracts task-dependent slots from perceptual features. SlotDiffusion [2] combines object-centric representations with diffusion decoding for images and videos. These works motivate structured representations but do not establish that the slots in our implementation discover objects.

Compute bottlenecks. Recurrent Interface Networks (RIN) [3] place substantial computation in latent tokens and use read/write attention to communicate with data tokens. This is a close architectural precedent. Our prototype instead tests a balanced transport assignment with mass-normalized aggregation and tied broadcast weights. The benefit of this constraint over RIN-style read/write attention remains unmeasured; the general bottleneck idea is not claimed as new.

Transformers and flows. DiT [4] motivates adaptive normalization in latent generative models. Flow Matching [5] motivates learning a time-conditioned vector field. Our code uses independent Gaussian-data pairs and a straight interpolation path; it does not solve a minibatch endpoint optimal-transport problem. Entropic optimal transport is used only inside the router, following the Sinkhorn framework [6].

Recent slot-space generation. SlotDiT [7], submitted on 15 September 2026, studies diffusion transformers in object-centric slot space. Our state variable remains the dense latent grid, with slots used internally. Any future novelty claim must compare the actual routing mechanism and experimental setting against this and earlier work.

3. Method

3.1 Data and conditioning contract

A frozen continuous tokenizer encodes video V into z_1 with shape $B \times C \times T_I \times H_I \times W_I$. Latent scaling s is explicit cache metadata rather than a borrowed constant. The same codec identity and scaling must be used when decoding generated samples. Let $N = T_I H_I W_I$ and let D be the feature width.

Flatten z into N tokens and apply a learned C-to-D projection. Add separable 3D sinusoidal positions for frame, row and column. This implementation uses additive positions, **not RoPE**. The implementation supports $D = 1024$ without requiring divisibility by six.

Conditions are either fixed zero vectors for an explicitly unconditional model, or cached vectors from an identified encoder with dimension D_c . A cached model requires a matching condition identity at inference. Random vectors are never substituted for unavailable text embeddings. A CLIP cache adapter is optional; using its 768-dimensional output requires $D_c = 768$ rather than silently truncating it.

3.2 Balanced feature routing

Let Q contain K learned anchors after projection and let X contain projected grid features. Unit-normalized anchor and grid vectors define a bounded cosine cost C_{ik} . We solve an entropy-regularized transport problem with uniform row and column masses:

$$P^* = \arg \min_{P \geq 0} \sum_{i,k} P_{ik} C_{ik} + \varepsilon \sum_{i,k} P_{ik} (\log P_{ik} - 1).$$

$$P \mathbf{1}_K = (1/N) \mathbf{1}_N, P^T \mathbf{1}_N = (1/K) \mathbf{1}_K.$$

Log-domain Sinkhorn iterations alternately normalize the two marginals. Define $A = NP$ at convergence. Its row sums are one and its column sums are N/K . This is a rectangular transport assignment, not a matrix whose row and column sums are both one.

The finite-iteration implementation performs a final row normalization, so row stochasticity holds up to floating-point error, while column balance is approximate. Both errors are measured; a fixed iteration count is not a convergence guarantee. Cosine normalization bounds the router scores, and log-domain reductions use at least float32 precision.

Aggregation uses normalized column weights:

$$E_k = (\sum_i A_{ik} X_i) / (\sum_i A_{ik} + \delta).$$

Balanced mass does not imply object segmentation. A uniform assignment satisfies the marginal constraints without separating objects. Equal capacity can also split large regions or combine small objects. Temporal identity and object semantics are empirical questions.

3.3 Slot transformer and broadcast

The transformer receives E , a sinusoidal flow-time embedding passed through an MLP, and the projected condition vector. Each block has its own AdaLN-Zero modulation and normalized attention/MLP branches. Final adaptive normalization is not zero-initialized, so time and conditioning can influence the initial vector field; this departure from a fully zero-output initialization is explicit.

Let F denote the resulting slot features. A linear output projection maps broadcast features into the original latent channel count C :

$$\hat{v}_i = W_o(\sum_k A_{ik} F_k) + b_o.$$

The output is reshaped back to $B \times C \times T_I \times H_I \times W_I$. There is no direct input-copy skip or zero-gated reconstruction shortcut. A is recomputed from the current noisy grid at every flow evaluation; generation does not require routing extracted from a clean target.

This tied broadcast is deliberately simple. For fixed A , the output lies in a restricted spatial span. A nonlinear spatial decoder or an independently learned write route could increase expressiveness but would introduce different costs and should be evaluated as separate variants.

3.4 Representation interventions

An optional intervention rescales one transformed slot before broadcast. This measures sensitivity to an internal feature group. Slot zero is not assumed to be background, and setting it to zero is not a physical velocity constraint. A semantic editing claim would require stable slot identification, localization metrics and controlled comparisons.

4. Learning and sampling

4.1 Optional masked warmup

The historical Phase 1 entry point now performs masked latent reconstruction. A randomly selected, nonempty proper subset of grid tokens is replaced with zero. Both the router and reconstruction head receive only the corrupted grid. Loss is evaluated on hidden target positions. The reconstruction head is separate from the velocity head, and there is no clean-input residual bypass.

This objective tests conditional reconstruction; it is not proof that all information is compressed into K vectors, because A remains an $N \times K$ side representation. The warmup is optional, and its effect on subsequent flow training must be ablated against training from scratch.

4.2 Latent-grid flow matching

Sample z_0 independently from a standard Gaussian, sample a cached data latent z_1 , and draw flow time τ uniformly from $[0,1]$:

$$z_\tau = (1 - \tau)z_0 + \tau z_1, u = z_1 - z_0.$$

$$L_{FM} = E[\|v_\theta(z_\tau, \tau, c) - u\|_2^2].$$

Encoding is performed once before caching; RGB noise is never repeatedly passed through the tokenizer during sampling. The input state, target velocity and predicted velocity occupy exactly the same latent space.

The router, input projection, slot transformer and velocity head train jointly on noisy grids. We do not freeze a router trained only on clean inputs and assume that it will transfer across noise levels. The old Phase 2

command is retained as an entry point, but its freeze policy has changed. Phase 3 is optional continuation of the same joint objective, not a separately demonstrated temporal-alignment method.

Optional assignment entropy and off-diagonal cosine-decorrelation penalties are exposed for ablation and default to zero. They provide biases, not semantic guarantees. K greater than D also makes exact pairwise orthogonality impossible in general.

4.3 Euler sampling and artifacts

Start from Gaussian latent noise. With S positive steps, evaluate the vector field at $\tau_j = j/S$ and update:

$$z_{j+1} = z_j + (1/S)v_\theta(z_j, j/S, c).$$

Save the generated latent with codec identity and scaling metadata. Video decoding is an explicit optional operation using matching pretrained weights. The program reports a video only after actually writing the decoded file. Old checkpoints are rejected by a versioned format check; model dimensions and attention head counts are loaded from checkpoint configuration.

5. Computational accounting

For L transformer blocks, D -dimensional grid/slot features and I Sinkhorn iterations, a simplified accounting excluding the frozen tokenizer is:

$$O(ND^2 + NKD + INK + LK^2D + LKD^2).$$

This includes grid key projections, aggregation/broadcast, marginal normalization, slot attention and feed-forward computation. Token and routing states require at least $O(ND + NK + KD)$ storage; training also retains intermediate activations. The implementation does not have K -only memory usage, and log-domain iteration intermediates may materially increase training memory.

Replacing a dense N -token attention core can reduce its quadratic term when K is small. It does not establish an equal reduction in full-model runtime or peak memory. Tokenizer work, projections, decoder quality, kernel overhead and optimizer state must be measured. No all-to-all communication factor or multi-GPU speedup is asserted. No distributed implementation is included.

6. Verification and proposed experiments

6.1 Completed correctness checks

The local implementation passed 26 CPU-only unit tests on 25 September 2026. These used synthetic tensors, at most tiny transformer configurations, and no video weights or training loops. Checks cover rectangular transport marginals, extreme-cost finiteness, numerical gradient checking, flow endpoints, output shapes, time/condition influence, nonzero router gradients, lack of an identity reconstruction path, Euler integration, deterministic sampling, cached-data validation, AdaLN initialization and checkpoint round trips.

These are engineering correctness results, not generation-quality results. Synthetic backward passes verify gradient availability without optimizer updates. They do not demonstrate convergence, object discovery, or physical reasoning. Real tokenizer integration, real-video decoding and accelerator performance remain unverified locally.

6.2 Experiments to conduct when resources permit

Use identical cached inputs, conditions, slot counts, transformer widths, optimization budgets and seeds to compare uniform aggregation, row-softmax routing and balanced Sinkhorn routing. All three routing options are implemented. Add an independent read/write attention baseline before claiming improvement over RIN-like designs.

A first controlled study can use externally prepared small synthetic sequences. Report held-out reconstruction/flow losses, transport residuals, slot utilization and identity-switch statistics where ground-truth identities exist. Compare optional masked warmup against no warmup. Vary K , transport temperature, iteration count, and regularization separately. A failed reconstruction or identity result is useful evidence about the bottleneck's limits.

For real-video experiments, separate training and held-out video sources; document codec checkpoints, compression ratios, scaling, frame stride, resolution, text encoder, seeds and training budget. Report generation quality, temporal behavior, peak allocated memory and measured throughput for matched models. Use multiple seeds and uncertainty estimates. If making physical claims, add tasks with observable collisions, occlusion and trajectories. Large-scale video training is not required on the author's local computer.

7. Limitations

There are no trained video-generation results in this revision. Slot semantics, temporal persistence, physical reasoning, text controllability and efficiency gains remain unproven. Balanced routing may be inappropriate for unequal object sizes, and low-rank tied broadcast may limit spatial detail. The $N \times K$ routing matrix stores layout information outside the slots. Cached conditioning loses any information omitted by its encoder. The baseline is not causal across video time merely because its tokenizer is causal. Long-sequence chunk continuity and distributed execution require additional design and evaluation.

The main deliverable is a clearer falsifiable hypothesis and a consistent implementation contract. Empirical studies must determine whether balanced routing earns its additional constraints and cost.

References

- [1] Locatello et al. Object-Centric Learning with Slot Attention. NeurIPS, 2020. <https://arxiv.org/abs/2006.15055>
- [2] Wu et al. SlotDiffusion: Object-Centric Generative Modeling with Diffusion Models. NeurIPS, 2023. <https://arxiv.org/abs/2305.11281>
- [3] Jabri, Fleet, and Chen. Scalable Adaptive Computation for Iterative Generation. ICML, 2023. <https://proceedings.mlr.press/v202/jabri23a.html>
- [4] Peebles and Xie. Scalable Diffusion Models with Transformers. ICCV, 2023. <https://arxiv.org/abs/2212.09748>
- [5] Lipman et al. Flow Matching for Generative Modeling. ICLR, 2023. <https://arxiv.org/abs/2210.02747>
- [6] Cuturi. Sinkhorn Distances: Lightspeed Computation of Optimal Transportation Distances. NeurIPS, 2013. <https://arxiv.org/abs/1306.0895>
- [7] Plepi and Behnke. SlotDiT: Object-Centric Representations for Diffusion Transformers. arXiv:2609.17414, 2026. <https://arxiv.org/abs/2609.17414>